

Data Analysis with Python 2024–2025

Parte 3: Mais NumPy

Processamento de Imagens

Tradução e Adaptação: University of Helsinki — MOOC.fi

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024–2025*, oferecido pela **The University of Helsinki MOOC Center**.

O que você vai aprender nesta página

- Aprender como processar imagens em Python.

Conteúdo

1 Processamento de Imagens

Nota sobre arquivos: Para acompanhar os exemplos desta seção e executar os códigos por conta própria, você precisará baixar ou escolher uma imagem qualquer (como uma foto JPG ou PNG) e salvá-la no mesmo diretório em que estiver rodando seus scripts Python. Recomendamos nomear a imagem como `painting.png` para corresponder aos exemplos abaixo.

Uma imagem é uma coleção de pixels, que é uma abreviação para elementos de imagem (*picture elements*). Uma imagem em tons de cinza pode ser representada como um array bidimensional, cujo primeiro eixo corresponde à coordenada x da imagem e o segundo eixo corresponde à coordenada y. O array contém, em cada par de coordenadas (x, y) , um valor que normalmente é um ponto flutuante entre 0.0 e 1.0, ou um inteiro entre 0 e 255. Isso especifica o tom de cinza. Por exemplo, se o array contém o valor 255 nas coordenadas (0,0), então na imagem o pixel no canto superior esquerdo é branco.

Em imagens coloridas, cada pixel tem três componentes de cor (ou canais) correspondentes ao vermelho, verde e azul (RGB). Esses canais formam o terceiro eixo da imagem, com cada canal armazenando um valor que representa a intensidade da respectiva cor. O valor do canal varia entre 0.0 e 1.0 (ou entre 0 e 255). Ao combinar diferentes valores de intensidade para os componentes vermelho, verde e azul de um pixel, é possível representar até 16,7 milhões de cores únicas.

Como as imagens podem ser representadas como arrays multidimensionais, podemos processá-las facilmente usando as funções do NumPy. Vejamos exemplos disso.

```
import matplotlib.pyplot as plt
import numpy as np

painting = plt.imread("painting.png")
print(painting.shape)
print(f"A imagem consiste em {painting.shape[0] * painting.shape[1]}
      pixels")
plt.imshow(painting)
plt.show()
```

(368, 640, 3)

A imagem consiste em 235520 pixels

Como a imagem agora é um array do NumPy, podemos executar algumas operações facilmente:

```
plt.imshow(painting[:,::-1]); # espelha a imagem na direcao x
```

A seguir, definimos os pixels nas primeiras 30 linhas como brancos:

```
painting2 = painting.copy() # nao estrague a pintura original!
painting2[0:30,:,:] = 1.0   # valor maximo para os tres componentes
                             produz branco
plt.imshow(painting2);
```

Para uma operação um pouco mais complicada, podemos criar uma função que retorna uma cópia da imagem de modo que ela escureça (fade para o preto) à medida que nos movemos para a esquerda.

```
def fadex(image):
    height, width = image.shape[:2]
    m = np.linspace(0, 1, width).reshape(1, width, 1)
    result = image * m      # note que contamos com o broadcasting aqui
    return result
```

```
modified = fadex(painting)
print(modified.shape)
plt.imshow(modified);
```

(368, 640, 3)

2 Encontrando clusters em uma imagem

Vamos primeiro gerar alguns dados para este exemplo:

```
n = 5
l = 256
im = np.zeros((l, l))
np.random.seed(0)
points = np.random.randint(0, l, (2, n**2)) # amostra n*n pixels
im[points[0], points[1]] = 1
plt.imshow(im);
```

```
from scipy import ndimage
im2 = ndimage.gaussian_filter(im, sigma=1/(8.*n)) # borra um pouco a
          imagem
plt.imshow(im2);
```

Vamos tentar encontrar clusters na imagem gerada acima:

```
mask = im2 > im2.mean() # mascara os pixels cuja intensidade esta acima
          da media
label_im, nb_labels = ndimage.label(mask) # componentes conectados
          formam clusters
print(f"0 numero de clusters e {nb_labels}")
plt.imshow(label_im);
```

0 numero de clusters e 12

Embora esse método que usamos tenha sido muito simples, ele ainda pode ser usado, por exemplo, para contar automaticamente o número de pássaros ou estrelas em uma imagem. Obviamente, humanos conseguem fazer isso facilmente, mas quando existem centenas ou milhares de imagens, é melhor usar máquinas para realizar esse trabalho mecânico.

Existe um grande número de aplicações de processamento de imagens, das quais listamos apenas algumas aqui:

- redução de ruído (*denoising*)
- correção de desfoque (*deblurring*)
- segmentação de imagens
- extração de características (*feature extraction*)
- zoom, rotação
- filtragem

3 Bibliotecas Adicionais

Existem várias bibliotecas escritas em Python que permitem o fácil processamento de imagens. Alguns exemplos destas:

- pillow
- scikit-image
- No Scipy, há o subpacote `ndimage` que também contém rotinas para processamento de imagens.

Créditos: Este conteúdo foi originalmente criado pela *The University of Helsinki MOOC Center* para o curso *Data Analysis with Python*.